

Progetto Multimedia Note Manager

Chen Daxin, mat. 2148627

Giugno 2026

Table of Contents

Progetto Multimedia Note Manager.....	1
Introduzione.....	2
Descrizione del modello.....	2
Polimorfismo.....	4
Persistenza dei dati.....	5
Funzionalità implementate.....	5
Rendicontazione delle ore.....	6

Introduzione

Multimedia Note Manager è un software desktop scritto in C++ sviluppato per gestire in un unico posto attività, scadenze e contenuti media. Usando il framework Qt, ho creato un sistema capace di trattare diversi tipi di note: da quelle testuali ai task con checklist, fino a note multimediali con immagini e promemoria temporizzati.

L'architettura del codice si basa su una gerarchia di classi che sfrutta l'ereditarietà virtuale e multipla. Questo serve a gestire le note "Avanzate", che devono ereditare funzionalità da più rami della struttura. Per la parte grafica, ho puntato sul polimorfismo: il metodo `getConfig()` permette all'interfaccia di capire autonomamente cosa mostrare (come anteprime o checkbox) in base al tipo di nota caricata. In questo modo il widget di visualizzazione non deve conoscere i dettagli della classe specifica, rendendo il codice più pulito. I dati vengono poi salvati in file JSON tramite le comuni finestre di dialogo.

L'obiettivo principale era prendere confidenza con le interfacce dinamiche di Qt. Gestire tipi di dati diversi in una griglia responsive mi ha permesso di mettere in pratica i concetti di incapsulamento, cercando di tenere il più possibile separata la logica di backend dalla visualizzazione.

Descrizione del modello

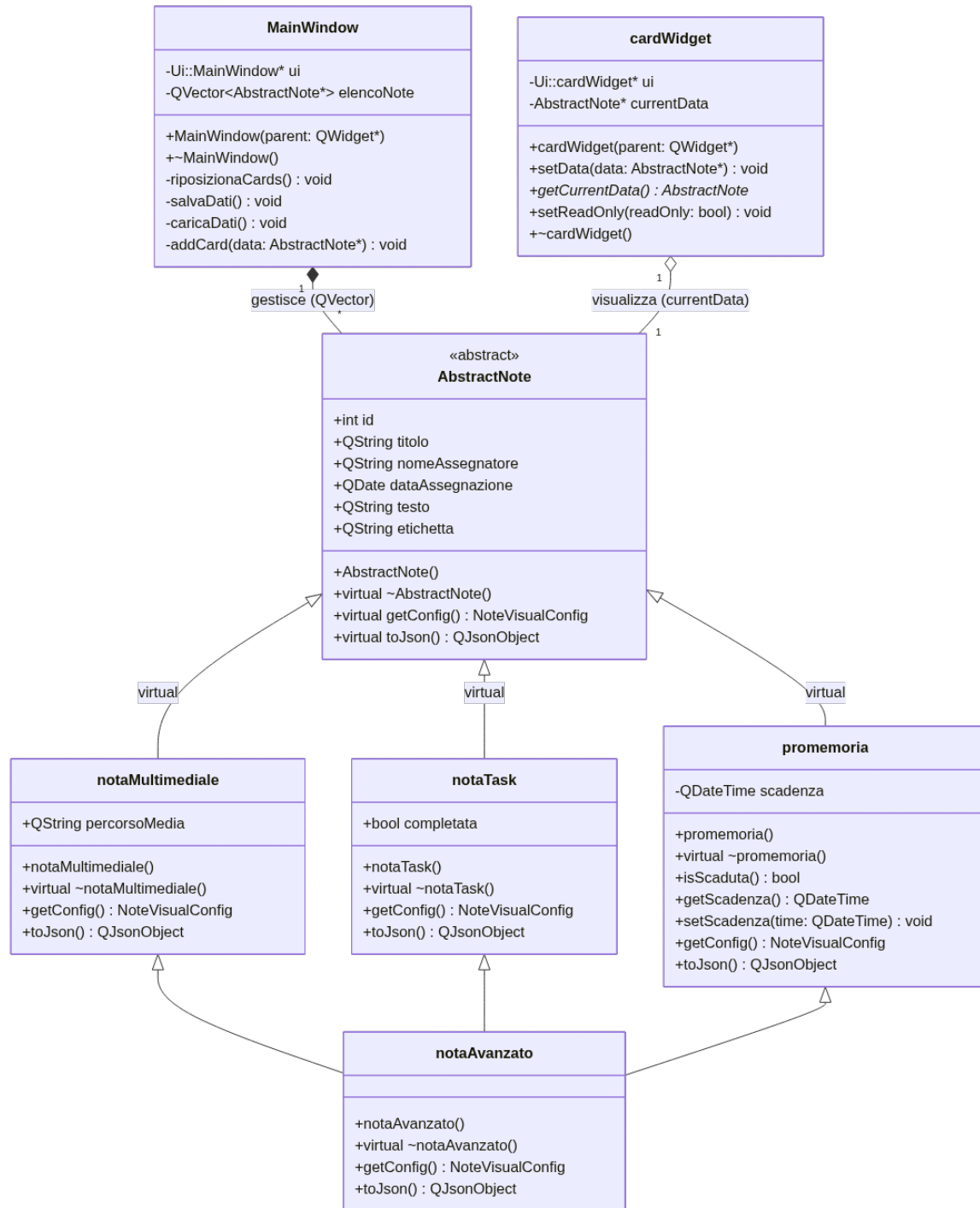
Il modello logico dell'applicazione organizza le informazioni personali attraverso una gerarchia di classi. La struttura separa la logica dei dati dall'interfaccia grafica: in questo modo, il modello gestisce le annotazioni, i promemoria e i file multimediali in modo indipendente dalla loro rappresentazione a video.

La classe base è `Nota`. Questa contiene gli attributi condivisi da tutti gli oggetti del sistema: un identificativo univoco (ID), un titolo, il testo del contenuto e un'etichetta per la categorizzazione. Il modello si articola poi in tre sottoclassi che aggiungono funzioni specifiche:

- `NotaMultimediale`: include l'attributo `percorsoMedia` per collegare immagini o file.
- `NotaTask`: gestisce lo stato di completamento di un'attività tramite un flag booleano.
- `Promemoria`: utilizza il tipo `QDateTime` per gestire le scadenze temporali.

Per combinare queste caratteristiche è stata definita la classe `NotaAvanzato`, che sfrutta l'ereditarietà multipla. Per evitare duplicazioni di dati o ambiguità (problema del diamante), le classi intermedie ereditano dalla base `Nota` in modalità virtuale. Questa soluzione assicura che ogni istanza di `NotaAvanzato` mantenga una sola copia dei campi base come ID e titolo, ottimizzando l'uso della memoria.

Il sistema gestisce il salvataggio dei dati tramite le classi JSON di Qt. Ogni elemento della gerarchia implementa il metodo `toJson()`, che mappa i propri attributi in un `JsonObject`. Questo metodo permette di salvare collezioni eterogenee di note in modo uniforme: ogni oggetto è responsabile della propria serializzazione su file. Infine, la struttura `NoteVisualConfig` fa da ponte verso la GUI, definendo i parametri di visualizzazione senza legare il modello ai widget specifici dell'interfaccia.



Il modello logico del progetto si articola a partire dalla classe base **nota**, che rappresenta le informazioni comuni a ogni attività personale gestibile dall'applicazione: identificativo univoco (id), titolo, contenuto testuale, etichetta di categorizzazione, nome dell'assegnatario e data di creazione. Da questa base si sviluppano tre sottoclassi concrete che utilizzano l'**ereditarietà virtuale** per permettere una specializzazione mirata dei dati: **notaMultimediale**, che aggiunge il supporto per file media tramite l'attributo `percorsoMedia`; **notaTask**, che integra un flag booleano per monitorare lo stato di completamento dell'attività; e **promemoria**, che estende le funzionalità con un attributo scadenza di tipo `QDateTime` per la gestione di impegni temporizzati.

Al vertice della gerarchia si trova la classe **notaAvanzato**, la quale eredita congiuntamente da tutte e tre le classi specializzate precedenti. L'uso dell'ereditarietà virtuale per le classi intermedie è una scelta architetturale per risolvere il

"problema del diamante", garantendo che un'istanza di `notaAvanzato` possieda un'unica copia degli attributi fondamentali della classe base `nota`, evitando ambiguità e ridondanza di memoria.

Poiché le classi del modello devono comunicare le proprie necessità di visualizzazione all'interfaccia grafica mantenendo una netta separazione tra logica e vista, si è scelto di non utilizzare un design pattern Visitor ma di affidarsi al polimorfismo diretto tramite il metodo `getConfig()`. Questo metodo restituisce una struttura `NoteVisualConfig` che agisce come descrittore per la GUI, indicando quali elementi grafici attivare (come checkbox, scadenze o anteprime immagini) in base al tipo dinamico dell'oggetto invocante. Parallelamente, la persistenza dei dati è gestita dal metodo polimorfo `toJson()`, che permette a ogni sottoclasse di mappare i propri attributi specifici in un oggetto `JsonObject`, includendo un campo "tipo" necessario alla successiva ricostruzione del tipo corretto durante il caricamento.

Infine, la logica di coordinamento è affidata alla classe **MainWindow**, che gestisce una collezione eterogenea di note memorizzate in un `QVector<nota*>`. La visualizzazione dei risultati è delegata alla classe **cardWidget**, la quale detiene un puntatore alla classe base e utilizza i metodi polimorfi descritti per configurare dinamicamente la propria interfaccia, garantendo che ogni tipo di nota sia presentato all'utente con i controlli e le informazioni più pertinenti.

Polimorfismo

In questo progetto, il polimorfismo è stato utilizzato in maniera non banale per garantire una netta separazione tra il modello logico e l'interfaccia grafica, evitando l'uso di costrutti rigidi come switch-case basati sul tipo o metodi impropri come `getType()`. L'utilizzo principale del polimorfismo risiede nella gestione della configurazione visiva e della serializzazione dei dati attraverso la gerarchia delle note.

Il metodo polimorfo più significativo è `getConfig()`. Ogni classe concreta della gerarchia (`notaMultimediale`, `notaTask`, `promemoria` e `notaAvanzato`) ridefinisce questo metodo per restituire una struttura `NoteVisualConfig` differente. Tale struttura contiene dei flag booleani (come `mostraScadenza`, `mostraImmagine`, `mostraTask`) che indicano quali componenti dell'interfaccia devono essere attivati per quel tipo specifico di nota. Il valore aggiunto di questo approccio è che il widget incaricato della visualizzazione, il `cardWidget`, può configurare i propri elementi grafici (nascondendo o mostrando pulsanti e label) invocando semplicemente `info->getConfig()` su un puntatore alla classe base `nota*`. Questo permette alla GUI di adattarsi dinamicamente al tipo di dato senza doverne conoscere l'identità concreta a runtime, facilitando l'estensibilità del sistema con nuovi tipi di note senza dover modificare il codice dei widget.

Un ulteriore utilizzo rilevante del polimorfismo riguarda la persistenza dei dati tramite il metodo `toJson()`. Ogni sottoclasse implementa la propria versione di questo metodo per mappare i propri attributi specifici in un oggetto `JsonObject`. Ad esempio:

- In **notaMultimediale**, il metodo include il percorso del file media.
- In **notaTask**, aggiunge lo stato di completamento dell'attività.
- In **notaAvanzato**, che sfrutta l'ereditarietà multipla virtuale, il metodo combina tutti i campi delle classi genitrici.

Una scelta implementativa di rilievo è rappresentata dalla classe **notaAvanzato**, che realizza una sintesi di tutte le funzionalità precedenti attraverso l'ereditarietà multipla. Per risolvere le problematiche strutturali legate a questa scelta (il cosiddetto "problema del diamante"), le classi intermedie `notaMultimediale`, `notaTask` e `promemoria` ereditano dalla classe base `nota` in modalità virtuale. Questo garantisce che un'istanza di `notaAvanzato` contenga una sola copia dei dati base (come titolo e ID), evitando ambiguità e spreco di memoria. Oltre alla gerarchia dei dati, il modello include una componente di servizio per la persistenza, basata sulle classi di Qt per la gestione del formato JSON. Ogni classe della gerarchia ridefinisce il metodo `toJson()`, responsabile di mappare gli attributi specifici dell'oggetto in un `JsonObject`. Questo approccio permette di gestire il salvataggio di una collezione eterogenea di note in modo uniforme, delegando a ogni oggetto la responsabilità di rappresentare correttamente il proprio stato per la memorizzazione su file. Infine, la comunicazione verso la GUI è mediata dalla struttura `NoteVisualConfig`, che permette al modello di descrivere le proprie necessità di visualizzazione senza dipendere direttamente dai widget grafici.

Grazie a questa implementazione, la classe `MainWindow` può iterare su un vettore eterogeneo di note (`QVector<nota*>`) e salvare l'intero database con un unico ciclo, delegando a ogni oggetto la responsabilità di rappresentare correttamente se stesso nel formato JSON. Questo approccio rispetta i principi di incapsulamento e fornisce una soluzione al problema del salvataggio di tipi di dati diversi in un unico file strutturato.

Con l'uso dell'ereditarietà virtuale per le classi intermedie `notaMultimediale` e `notaTask` permette l'utilizzo del polimorfismo corretto nella classe `notaAvanzato`, risolvendo le ambiguità strutturali e garantendo che i metodi polimorfi operino su un'unica istanza condivisa della classe base.

Persistenza dei dati

La persistenza dei dati è realizzata attraverso l'uso del formato JSON ottima per la sua capacità di rappresentare strutture gerarchiche complesse in modo leggibile e la sua integrazione nativa a Qt. Tutte le note gestite dall'applicazione vengono salvate in un unico file locale che contiene un array di oggetti.

Il processo di salvataggio sfrutta il polimorfismo: la classe `MainWindow` itera sulla collezione di note e invoca per ciascuna il metodo virtuale `toJson()`. Ogni sottoclasse ridefinisce questo metodo per mappare i propri attributi specifici; ad esempio, `notaTask` salva lo stato "completata", mentre `notaAvanzato` include anche i dati relativi al "percorsoMedia" e alla "scadenza". Per permettere la corretta ricostruzione degli oggetti in fase di caricamento (deserializzazione), a ogni oggetto viene aggiunto un attributo "tipo" (es. "task", "avanzato", "promemoria") che funge da discriminatore per istanziare la classe corretta.

Coerentemente con i vincoli del progetto, l'applicazione gestisce il salvataggio e il caricamento esclusivamente tramite finestre di dialogo grafiche (`QFileDialog`), evitando l'utilizzo di percorsi di file cablati nel codice. Tra i file del progetto è incluso un file d'esempio chiamato `Salvataggio.json`. Questo file contiene un'istanza per ogni tipologia di nota implementata, permettendo di verificare come i diversi attributi (come date, stati booleani o percorsi di file) vengano tradotti correttamente nel formato di interscambio.

Funzionalità implementate

L'interfaccia grafica dell'applicazione è stata progettata per essere immediata, permettendo all'utente di gestire i propri impegni in modo naturale. Tuttavia, il sistema include diverse accortezze tecniche e funzionalità di usabilità che, pur non essendo sempre evidenti, potrebbe migliorare l'esperienza d'uso e la robustezza del software. Ad esempio, il programma implementa scorciatoie da tastiera per le operazioni frequenti e richiede una conferma esplicita prima della chiusura della finestra o dell'eliminazione di una nota, prevenendo la perdita accidentale di dati.

Le funzionalità sono state suddivise per chiarezza in due categorie: funzionali ed estetiche.

Le funzionalità funzionali comprendono:

- **Gestione di quattro tipologie di note:** il sistema supporta note multimediali, task, promemoria e note avanzate, ciascuna con logiche di visualizzazione e dati specifici. voto finale 3
- **Persistenza polimorfa in formato JSON:** salvataggio e caricamento dell'intero database tramite finestre di dialogo, con gestione automatica del tipo dinamico degli oggetti.
- **Sistema di filtraggio per categoria:** possibilità di visualizzare selettivamente solo determinati tipi di note (es. solo i Task o solo i Promemoria) tramite la barra laterale.
- **Gestione dinamica dei Tag:** implementazione di un sistema di etichette personalizzabili per organizzare e filtrare ulteriormente i contenuti.
- **Verifica delle scadenze:** logica interna per determinare se un promemoria è scaduto rispetto alla data attuale.

Le funzionalità grafiche e di usabilità comprendono:

- **Layout Responsive:** il sistema calcola dinamicamente il numero di colonne della griglia principale in base alla larghezza della finestra, riposizionando le "card" in tempo reale durante il ridimensionamento.
- **Sidebar di navigazione a comparsa:** un pannello laterale (nascondibile tramite pulsante) che facilita la navigazione tra le diverse categorie di note.
- **Barra dei menù e Toolbar:** accesso rapido alle funzioni di file (Salva, Carica, Esci) con icone identificative e indicazione delle scorciatoie da tastiera.
- **Configurazione polimorfa dell'interfaccia:** ogni card si configura graficamente in modo automatico (mostrando o nascondendo elementi come checkbox o anteprime immagini) in base alla configurazione fornita dal modello logico.
- **Navigazione a schermo:** utilizzo di un'interfaccia basata su `QStackedWidget` per passare fluidamente dalla schermata di avvio alla dashboard principale.
- **Editor integrato:** pannello dedicato per la creazione e modifica delle note che si adatta dinamicamente ai campi necessari per il tipo di nota selezionato.

- **Feedback visivo e messaggistica:** utilizzo di finestre di dialogo informative (QMessageBox) per confermare azioni critiche o segnalare errori all'utente.

Per migliorare l'usabilità e l'efficienza nella navigazione, come suggerito dalle specifiche di progetto per arricchire l'esperienza utente, sono state implementate scorciatoie da tastiera usando la classe QShortcut. Queste permettono di richiamare le funzioni principali dell'applicazione in modo rapido senza l'utilizzo del mouse:

- **Alt+B:** gestisce la visualizzazione della sidebar laterale, permettendo di mostrarla o nascondere rapidamente per massimizzare lo spazio di lavoro.
- **Alt+E:** focalizza l'attenzione sulla gestione delle etichette (label), facilitando l'organizzazione categoriale delle note.
- **Alt+S:** attiva istantaneamente la funzione di ricerca (Search), aprendo l'overlay dedicato per filtrare i contenuti.
- **Alt+A:** avvia la procedura per aggiungere una nuova nota, portando l'utente direttamente ai campi di inserimento.

Rendicontazione delle ore

In questa sezione vengono riportate le attività svolte per la realizzazione del progetto, confrontando le ore preventivate con quelle effettivamente impiegate. Come indicato nelle specifiche, il progetto è stato stimato per richiedere circa **40 ore** di lavoro individuale.

Attività	Ore Previste	Ore Effettive
Studio e progettazione del modello logico (UML e Gerarchia)	8	10
Studio del framework Qt (Signal/Slot e Layout dinamici)	8	10
Sviluppo del codice del modello e logica polimorfa	10	12
Sviluppo della GUI (MainWindow, Card e Pannelli di editing)	12	14
Implementazione persistenza JSON e gestione file	5	6
Test, debugging e rifiniture grafiche	5	7
Stesura della relazione tecnica	2	3
Totale	50	62

Il monte ore effettivo ha superato quello previsto di circa il **24%**. Tale scostamento è motivato principalmente dalle seguenti sfide tecniche incontrate durante lo sviluppo:

- **Complessità dell'Ereditarietà:** L'implementazione della classe notaAvanzato tramite ereditarietà multipla virtuale per risolvere il "problema del diamante" ha richiesto tempi di progettazione e debug superiori al previsto per assicurare che i metodi polimorfi operassero correttamente sui dati condivisi.
- **Gestione Dinamica della GUI:** Lo sviluppo di un'interfaccia responsive capace di ricalcolare le colonne e riposizionare le card in tempo reale durante il ridimensionamento della finestra ha comportato uno studio approfondito degli eventi di Qt (resizeEvent) e della gestione manuale dei layout all'interno della griglia.
- **Polimorfismo e Configurazione:** La messa a punto del sistema di configurazione polimorfa tramite il metodo getConfig(), pur garantendo una separazione netta tra modello e vista, ha richiesto diversi cicli di test per assicurare che ogni widget (cardWidget) mostrasse correttamente solo i campi pertinenti al tipo dinamico della nota.
- **Persistenza dei Dati:** La gestione della serializzazione e deserializzazione di oggetti eterogenei in formato JSON ha richiesto aggiuntivi per garantire che l'attributo discriminatore del tipo permettesse la ricostruzione fedele della gerarchia delle note al caricamento del file.